

Tab 1

OnTime

Public Transport Intelligence System

Software Requirements Specification & Business Plan

Document Version	v2.0 — Final
Date	April 2025
Classification	Confidential — Internal Use Only
MVP Corridor	Moratuwa — Kadawatha, Sri Lanka

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the OnTime Public Transport Intelligence System. The document serves as the authoritative reference for all development teams (G1 through G4), stakeholders, and quality assurance processes throughout the project lifecycle.

This document has been prepared in accordance with IEEE 830-1998 standards for SRS documentation and integrates a business plan to support investment readiness and commercial deployment.

1.2 Scope & Project Overview

OnTime is a comprehensive, IoT-enabled public transport management platform designed to address the chronic unreliability of urban bus services in Sri Lanka. The system delivers real-time GPS tracking, machine learning-driven Estimated Time of Arrival (ETA) predictions, automated fleet anomaly detection, crowd density sensing, and intelligent trip lifecycle management.

The MVP operational corridor covers the Moratuwa-to-Kadawatha route — one of the highest-density commuter corridors in the Western Province of Sri Lanka. The platform is architected for horizontal scaling to additional routes and districts post-MVP validation.

1.3 System Architecture Overview

The OnTime system is organized into four interdependent engineering subgroups:

Group	Domain	Responsibilities
G1	IoT & Edge	Arduino Nano hardware, GPS acquisition, crowd sensing, edge processing, offline buffering
G2	Data & Intelligence	Kafka/MQTT data ingestion, ML ETA prediction, anomaly detection, fleet management APIs
G3	Frontend	Passenger mobile app, Driver mobile app, Admin web dashboard
G4	Platform & Security	Cloud infrastructure, Kubernetes orchestration, Kafka/MQTT broker, Keycloak authentication

1.4 Definitions, Acronyms & Abbreviations

Term / Acronym	Definition
ETA	Estimated Time of Arrival — the predicted time a bus will reach a downstream stop
IoT	Internet of Things — the network of embedded devices transmitting data
MQTT	Message Queuing Telemetry Transport — lightweight IoT messaging protocol

Kafka	Apache Kafka — distributed event streaming platform for high-throughput data ingestion
JWT	JSON Web Token — stateless authentication token used for Driver/Admin sessions
SRS	Software Requirements Specification — this document
FR	Functional Requirement — a specific capability the system must provide
NFR	Non-Functional Requirement — a quality attribute or constraint on the system
MVP	Minimum Viable Product — the initial deployment scope (Moratuwa–Kadawatha corridor)
XGBoost	Extreme Gradient Boosting — ML algorithm used for ETA regression prediction
DXA	Device-independent unit (1 inch = 1440 DXA) used in document formatting
Keycloak	Open-source Identity and Access Management (IAM) solution used for authentication
2G/GSM	Second Generation cellular network used for IoT hardware data transmission
IR	Infrared — sensor technology used for passenger counting at bus doors
EEPROM	Electrically Erasable Programmable Read-Only Memory — used for offline data buffering

1.5 Document Conventions

Requirements are identified using the convention [Group]-[Category]-[Number], e.g., FR-G1.1. Non-functional requirements use the format NFR-[Category]-[Number]. All requirements are atomic, verifiable, and traceable to at least one test case.

1.6 References

- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications
- Arduino Nano v3 Datasheet & ATmega328P Reference Manual
- u-blox NEO-6M GPS Module Datasheet
- SIMCom SIM800L AT Command Manual v1.09
- Apache Kafka 3.x Documentation — kafka.apache.org
- Eclipse Mosquitto MQTT Broker Documentation
- Keycloak 22.x Identity & Access Management Documentation
- XGBoost Documentation — xgboost.readthedocs.io

2. Business Plan

2.1 Executive Summary

OnTime is a B2G (Business-to-Government) and B2C (Business-to-Consumer) SaaS platform targeting the public transport sector in Sri Lanka. By deploying low-cost IoT hardware on existing bus fleets and providing a cloud-based intelligence layer, OnTime transforms unreliable, information-dark bus networks into predictable, data-driven services.

The immediate addressable market is the National Transport Commission (NTC)-regulated intercity and intra-provincial bus network, which operates over 22,000 buses nationally. The MVP targets the Moratuwa-to-Kadawatha corridor — a 35 km route handling approximately 85,000 daily passenger trips.

2.2 Problem Statement

Public bus commuters in Sri Lanka face three critical pain points:

- **Unpredictability:** No real-time bus location or ETA information is publicly available, forcing commuters to wait at stops with no visibility.
- **Overcrowding:** Crowd density information is unavailable, leading to poor distribution of passenger load across buses.
- **Operational Inefficiency:** Fleet operators have no live data to manage breakdowns, route deviations, or schedule adherence.

These problems result in an estimated 15–20% productivity loss for daily commuters and contribute to declining public transport ridership in favor of private vehicles, worsening urban congestion.

2.3 Solution

OnTime addresses these pain points through a three-layer solution architecture:

- **Hardware Layer (G1):** Low-cost, ruggedized IoT hardware units (BOM cost < LKR 8,000 per unit) installed on buses. Each unit transmits GPS and crowd data in real time.
- **Intelligence Layer (G2):** Cloud-based ML pipelines that convert raw GPS streams into accurate ETAs, detect anomalies, and manage fleet state in real time.
- **Interface Layer (G3):** Intuitive mobile and web applications for passengers, drivers, and operations administrators.

2.4 Market Analysis

2.4.1 Target Market Segments

Segment	Size (Estimated)	Value Proposition
Government / NTC	1 regulator, 22,000+ buses	Fleet oversight, compliance reporting, accident reduction
Private Bus Operators	~4,500 operators nationally	Reduced fuel waste via optimized dispatch, driver accountability
Daily Commuters	~3.2 million daily (NTC)	Live ETAs, crowd avoidance, time savings

Advertisers	FMCG, fintech, retail brands	Geo-targeted advertising to captive commuter audience
--------------------	------------------------------	---

2.4.2 Competitive Landscape

Competitor	Live GPS	ML ETA	Crowd Data	Local Focus
Google Maps Transit	Limited	Yes	No	No
Moovit	Partial	Yes	No	No
Local Apps (Pick.Ik etc.)	No	No	No	Partial
OnTime (Proposed)	Yes (1 Hz)	Yes (XGBoost)	Yes (IoT)	Yes (MVP)

2.5 Revenue Model

OnTime employs a multi-stream SaaS revenue model:

Revenue Stream	Model	Target
Government/NTC Licensing	Annual SaaS License	LKR 5M–15M/year per district
Operator Subscription	Per-bus monthly fee	LKR 1,500–2,500 per bus/month
Passenger App (Premium)	Freemium	Ad-free, saved routes, advanced alerts
Ticketing Commission	Transaction fee (optional module)	1.5–2% per fare transaction
In-App Advertising	CPM / targeted ads	FMCG and fintech brands

2.6 Financial Projections (3-Year)

Metric	Year 1 (MVP)	Year 2	Year 3
Buses Equipped	50	400	2,000
Hardware Revenue (LKR M)	0.4	3.2	16.0
SaaS / Subscription Revenue (LKR M)	1.5	9.0	42.0
Total Revenue (LKR M)	1.9	12.2	58.0
Estimated Operating Cost (LKR M)	4.8	7.2	18.0
EBITDA (LKR M)	(2.9)	5.0	40.0

Note: Year 1 deficit reflects MVP build and hardware subsidization costs. Break-even is projected mid-Year 2 following provincial expansion.

2.7 Go-to-Market Strategy

Phase 1 — MVP Pilot (Months 1–6)

- Deploy 50 IoT units on selected Moratuwa–Kadawatha buses in partnership with 2–3 cooperative operators.
- Soft-launch passenger app with free GPS tracking and ETA features.
- Collect performance data, user feedback, and system telemetry to validate ML models.

Phase 2 — Provincial Expansion (Months 7–18)

- Scale to 400 buses across Western Province in partnership with NTC.
- Launch driver and admin portals with full fleet management suite.
- Introduce optional ticketing module and in-app advertising.

Phase 3 — National Rollout (Months 19–36)

- Expand to Central, Southern, and North Western provinces.
- Pursue formal government licensing agreements and API partnerships with ride-hailing platforms.
- Explore export to South Asian markets (Bangladesh, Myanmar) with similar transport infrastructure.

2.8 Risk Register

Risk	Likelihood	Impact	Mitigation
2G network degradation	Medium	High	Offline buffering (SD/EEPROM); 4G upgrade path planned
Operator hardware resistance	Medium	High	Subsidized hardware, demonstrable ROI via fuel savings dashboards
GPS signal loss in tunnels/urban canyons	Medium	Medium	Kalman filtering; dead reckoning fallback via speed/heading
Regulatory data privacy requirements	Low	Medium	Anonymized passenger data; GDPR-aligned privacy policy
ML model drift over time	High	Medium	Monthly model retraining pipeline; A/B testing of model versions
Competitive entry by Google/Moovit	Low	High	Local hardware ownership, government partnerships as moat

3. User Classes & Characteristics

The OnTime system serves three primary user roles, each with distinct access levels, authentication requirements, and interaction patterns.

User Role	Auth Required	Primary Interface	Key Responsibilities
Passenger	Optional (ticketing only)	Mobile App (iOS/Android)	View live bus tracking, ETAs, crowd levels, route search
Bus Driver	Mandatory (JWT via Keycloak)	Mobile App (Android)	Manage trip states, report incidents, override crowd levels
Administrator	Mandatory (JWT via Keycloak)	Web Dashboard	Dispatch buses, monitor fleet anomalies, manage schedules

3.1 Passenger

Passengers are the primary end-users of the mobile application. The app is available without registration for core tracking and ETA features. Passenger interactions are read-only with respect to fleet data. Passengers may optionally authenticate to access the wallet and ticketing module (stretch goal).

3.2 Bus Driver

Drivers operate the bus-side mobile application installed on a device mounted or carried in the cab. Drivers authenticate using credentials provisioned by an Administrator. The driver app is the primary interface for reporting trip state transitions and incidents, and for manually overriding sensor-derived crowd readings when sensors malfunction.

3.3 Administrator (Fleet Operations)

Administrators are operations staff employed by bus operators or the NTC. They manage the complete fleet lifecycle via the web dashboard: assigning buses to departure slots, monitoring live fleet health, acknowledging anomaly alerts, and managing user accounts for drivers.

4. Functional Requirements

This section details all functional requirements organized by engineering subgroup. Each requirement is uniquely identified, atomic, and verifiable.

4.1 Hardware & Edge Computing (G1)

FR-G1.1 — GPS Data Acquisition

The Arduino Nano firmware shall capture geographic coordinates from the onboard u-blox NEO-6M GPS module. The firmware shall extract latitude, longitude, altitude, speed-over-ground, number of locked satellites, and UTC timestamp from the NMEA data stream. Data shall be acquired at a minimum frequency of 1 Hz.

- Input: NMEA sentence stream from NEO-6M over SoftwareSerial
- Output: Validated GPS fix object with all fields populated
- Acceptance Criterion: Fix acquired within 60 seconds in open-sky conditions; stale fixes (> 5 seconds old) must be discarded

FR-G1.2 — Crowd Density Sensing

The hardware shall deploy dual sensors (infrared or ultrasonic) positioned at the bus entry and exit doors. The firmware shall implement a bidirectional passenger counter that increments on ingress and decrements on egress. The current occupancy count shall be mapped to a crowd state enumeration.

State	Threshold	Indicator Color
NOT_FULL	< 60% capacity	Green
SEMI_FULL	60–90% capacity	Yellow
FULL	> 90% capacity	Red

FR-G1.3 — Edge Signal Processing

Prior to transmission, the firmware shall apply a local smoothing algorithm to reduce GPS coordinate noise. A Kalman filter or exponential moving average with a configurable window size shall be implemented. The firmware shall also detect and discard clearly erroneous coordinates (e.g., coordinates outside the island of Sri Lanka's bounding box, or speed readings > 120 km/h for urban buses).

FR-G1.4 — Offline Buffering & Reconnection

In the event of cellular network unavailability, the firmware shall serialize GPS payloads as JSON objects and persist them to either an SD card module (if present) or the MCU EEPROM (capacity-limited). Upon cellular reconnection, the firmware shall publish buffered payloads in chronological order before resuming live transmission. Buffered payloads shall be clearly timestamped for accurate back-fill processing.

FR-G1.5 — Hardware Payload Transmission

The firmware shall transmit validated, smoothed GPS and crowd data over the 2G cellular network using the SIM800L module via the MQTT protocol. The transmission payload shall conform to the schema defined in Section 6 of this document.

4.2 Data Ingestion & Intelligence (G2)

FR-G2.1 — Real-Time Data Ingestion

The backend shall operate an MQTT-to-Kafka bridge that subscribes to all bus telemetry topics (transport/bus/+/location) and forwards validated payloads to the internal Kafka topic `gps.raw.{bus_id}`. The bridge shall consume data at a target rate of 1 Hz per bus and must support concurrent ingestion for all active buses on a route.

FR-G2.2 — ETA Prediction Engine

The backend shall host an XGBoost regression model that consumes the real-time GPS stream and outputs the predicted remaining travel time (in seconds) to each downstream stop on the route. Predictions shall refresh with each GPS update. The model shall be trained on historical trip data segmented by time-of-day, day-of-week, weather conditions, and known traffic bottlenecks.

- Input Features: Current GPS coordinates, current speed, time of day, day of week, distance to stop, historical travel time for segment
- Output: Predicted seconds to each downstream stop
- Accuracy Target: Mean Absolute Error (MAE) < 90 seconds under normal operating conditions

FR-G2.3 — Crowd State Pipeline

The backend shall ingest the `crowdStatus` field from each hardware payload and update the corresponding bus record in the real-time state database. The current crowd state for each bus shall be included in the WebSocket feed consumed by frontend clients. Drivers shall be able to submit a manual crowd override via the Driver API, which shall supersede sensor-derived values until the next automatic sensor reading is received.

FR-G2.4 — Fleet Anomaly Detection

The backend shall implement a three-layer anomaly detection pipeline to identify and flag abnormal fleet events:

- Layer 1 — Statistical Detection: Flag buses with GPS speed or positional deviation exceeding 3 standard deviations from the historical mean for that route segment and time window.
- Layer 2 — Isolation Forest: Apply a trained Isolation Forest model to detect multi-variate anomalies in the speed, heading, and stop-dwell-time feature space.
- Layer 3 — Business Rules Engine: Apply deterministic rules including: (a) bus stopped for > 10 minutes outside a designated stop, (b) bus trajectory deviating > 500 metres from the route polyline, (c) no GPS update received for > 3 minutes.

All detected anomalies shall generate an alert payload that is published to the Admin dashboard via WebSocket and persisted to the anomaly log database.

4.3 Fleet Management (G2 & G3)

FR-G3.1 — Departure Slot Management

The system shall maintain a pre-defined grid of departure time slots for each route. An Administrator shall be able to assign an available, registered bus to an upcoming departure slot via the web dashboard. The system shall enforce that each slot may only be assigned one bus. Unassigned slots shall be visible to the Admin as requiring attention.

FR-G3.2 — Bus Trip State Machine

The system shall enforce the following trip state machine. State transitions shall only be permitted in the defined sequence and shall be triggered by the Driver via the Driver app or automatically by the backend.

From State	To State	Trigger
WAITING_AT_DEPOT	DEPARTED_ORIGIN	Driver taps 'Start Trip' in app
DEPARTED_ORIGIN	EN_ROUTE	GPS confirms movement from origin stop
EN_ROUTE	ARRIVED_DESTINATION	GPS geo-fence trigger at destination
EN_ROUTE	INCIDENT_REPORTED	Driver reports breakdown, accident, or traffic
INCIDENT_REPORTED	EN_ROUTE	Driver resolves incident, Admin acknowledges
ARRIVED_DESTINATION	WAITING_AT_DEPOT	Admin resets bus for next assignment

FR-G3.3 — Incident Reporting

The Driver app shall provide a dedicated incident reporting interface. Upon submitting a report, the Driver shall categorize the incident from a predefined list. The report shall automatically transition the bus to INCIDENT_REPORTED state and generate an Admin alert.

Incident Code	Description
BREAKDOWN	Mechanical failure — bus cannot proceed
ACCIDENT	Collision or safety incident
HEAVY_TRAFFIC	Severe congestion causing major delay
ROAD_CLOSURE	Route blocked requiring diversion
MEDICAL_EMERGENCY	Passenger or driver medical incident

4.4 Frontend Interfaces (G3)

FR-G4.1 — Passenger Live Tracking & Crowd Display

The passenger mobile application shall render a live map view displaying all active buses on the selected route. Bus position markers shall update in real time via WebSocket feed with a visible lag of no more than 5 seconds. Each bus marker or associated information card shall display the crowd level using a clear colour-coded visual indicator (Green / Yellow / Red as defined in FR-G1.2). The estimated arrival time for the next stop shall be displayed alongside the bus marker.

FR-G4.2 — Route Search & Discovery

The passenger app shall provide a search interface allowing users to find routes by route number, origin stop name, or destination stop name. Search results shall display the next scheduled

departure, the current crowd level of the approaching bus, and the predicted ETA to the user's selected stop.

FR-G4.3 — Admin Operations Dashboard

The Admin web dashboard shall provide authenticated administrators with the following capabilities:

- Live fleet map displaying all active buses with real-time position, state, and crowd level
- Anomaly alert panel with audible/visual notification for new alerts and one-click acknowledgment
- Departure slot grid with bus assignment interface
- Driver and bus registration management
- Historical trip reports and performance analytics

FR-G4.4 — Driver Trip Management App

The Driver mobile application shall provide authenticated drivers with the following capabilities:

- Trip state transition controls (Start Trip, Report Incident, End Trip)
- Current route and assigned departure slot details
- Manual crowd level override interface (active only when sensor malfunction is reported)
- Real-time Admin notifications and messages

4.5 Optional Module: Integrated Ticketing (Stretch Goal — Low Priority)

The following requirements shall be implemented only after all core tracking, ETA, and fleet management features are fully functional and validated.

FR-Opt.1 — Passenger Digital Wallet

The passenger app shall allow registered users to create an account, top up a virtual wallet balance via a payment gateway, and generate a session-unique dynamic QR code that encodes the passenger's user ID and a cryptographic token. The QR code shall expire after 90 seconds and regenerate automatically.

FR-Opt.2 — Driver/Conductor Fare Validation

The Driver app shall include an embedded QR code scanner. Upon boarding scan, the system shall record the boarding stop. Upon alighting scan, the system shall calculate the applicable fare based on the distance between boarding and alighting stops according to the NTC fare matrix. The passenger's wallet balance shall be debited accordingly.

FR-Opt.3 — Transaction API

The G2 backend shall expose secured REST endpoints for fare deduction and transaction history. All transaction endpoints shall require JWT authentication. Transaction records shall be persisted with full audit trail including timestamp, bus ID, route, boarding stop, alighting stop, fare amount, and resulting wallet balance.

5. External Interfaces & API Contracts

5.1 Hardware to Backend (G1 → G2 via G4 MQTT Broker)

Attribute	Specification
Protocol	MQTT v3.1.1 over TCP/TLS
Topic Pattern	transport/bus/{busId}/location
QoS Level	QoS 1 (at-least-once delivery)
Frequency	1 Hz (one message per second per active bus)
Authentication	X-API-Key header validation at MQTT broker level
Encoding	UTF-8 JSON

5.1.1 GPS Telemetry Payload Schema

Field	Type	Required	Description
busId	string	Yes	Unique bus identifier (e.g., G-BUS-01)
routeId	string	Yes	Assigned route identifier
lat	float	Yes	WGS84 latitude (decimal degrees)
lng	float	Yes	WGS84 longitude (decimal degrees)
speed	float	Yes	Speed over ground in km/h
crowdStatus	enum	Yes	One of: NOT_FULL, SEMI_FULL, FULL
satellites	integer	Yes	Number of GPS satellites locked (minimum 4 for valid fix)
deviation	boolean	Yes	True if bus has deviated from expected route polyline
timestamp	string	Yes	ISO 8601 UTC timestamp (e.g., 2025-04-06T08:23:11Z)

5.2 Backend REST API (G2 → G3)

The backend shall expose a RESTful API over HTTPS with JSON request/response bodies. All authenticated endpoints shall validate a JWT Bearer token issued by Keycloak.

Method	Endpoint	Description	Auth	Response
GET	/api/v1/routes	List all routes	None	Array of route objects
GET	/api/v1/buses/live	Get live positions for all active buses	None	Array of BusStatus objects

GET	/api/v1/buses/{id}/eta	Get ETA predictions for a specific bus	None	Array of {stopId, etaSeconds}
POST	/api/v1/trips/{id}/state	Transition bus trip state	Driver JWT	Updated TripState object
POST	/api/v1/trips/{id}/incident	Report a trip incident	Driver JWT	Incident acknowledgment object
POST	/api/v1/admin/slots/{id}/assign	Assign a bus to a departure slot	Admin JWT	Updated Slot object
GET	/api/v1/admin/anomalies	List all unresolved anomaly alerts	Admin JWT	Array of AnomalyAlert objects

5.3 Real-Time WebSocket Feed (G2 → G3)

The backend shall expose a WebSocket endpoint at `wss://api.ontime.lk/v1/live` for real-time fleet state streaming. Upon connection, the server shall push the full current fleet state. Subsequently, delta updates shall be pushed on every GPS update received (approximately 1 Hz per bus). The WebSocket feed shall not require authentication for passenger-facing data; admin-level data fields shall require a valid token in the connection handshake.

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Measurement Criterion
NFR-P-01	End-to-end GPS data to API delivery latency must be ≤ 2.0 seconds	Measured from hardware timestamp to WebSocket client delivery
NFR-P-02	Frontend map update lag must remain < 5 seconds behind real-world bus position	Load tested with 50 concurrent passenger clients
NFR-P-03	ETA prediction computation time must be < 200 milliseconds per bus per update cycle	95th percentile measured in production load conditions
NFR-P-04	The REST API must handle a minimum of 500 concurrent requests at $< 300\text{ms}$ p95 latency	Load test with Apache JMeter or k6
NFR-P-05	ML model MAE for ETA prediction must remain < 90 seconds under normal conditions	Continuous evaluation against ground truth arrival logs

6.2 Hardware Constraints

ID	Requirement	Rationale
NFR-H-01	Firmware must operate within the Arduino Nano's 2KB SRAM constraint	Dynamic memory allocation must not exceed 70% of SRAM to prevent heap/stack collision
NFR-H-02	SoftwareSerial usage must be carefully managed — only one SoftwareSerial port active at a time	Arduino Nano has a single hardware UART; GPS and GSM modules share SoftwareSerial
NFR-H-03	Hardware unit must operate reliably in ambient temperature range of 5°C to 60°C	Accounts for bus interior temperatures in Sri Lankan climate
NFR-H-04	Mean Time Between Failures (MTBF) for hardware unit must exceed 6 months of continuous operation	Minimizes maintenance overhead during MVP phase
NFR-H-05	Bill of Materials (BOM) cost per unit must not exceed LKR 8,000	Required to maintain operator affordability and breakeven analysis targets

6.3 Reliability & Availability

ID	Requirement
NFR-R-01	The MQTT bridge must automatically reconnect within 10 seconds of a connection drop with exponential backoff.
NFR-R-02	The cloud backend must maintain a minimum uptime of 99.5% (excluding scheduled maintenance windows).

NFR-R-03	All database writes must use transactions to prevent partial state corruption in the event of service failure.
NFR-R-04	The offline buffering system on the hardware must be capable of storing a minimum of 30 minutes of GPS data during connectivity loss.

6.4 Security Requirements

ID	Requirement
NFR-S-01	GPS data ingestion endpoints must enforce strict X-API-Key header authentication. Requests without a valid key must return HTTP 401.
NFR-S-02	All Admin and Driver actions via the REST API must be protected by Keycloak JWT validation. Tokens must be validated for signature, expiry, and role claims.
NFR-S-03	All data in transit must be encrypted using TLS 1.2 or higher across all communication channels (MQTT, WebSocket, REST).
NFR-S-04	Passenger personally identifiable information (PII) must not be logged in system telemetry or error logs.
NFR-S-05	Admin and Driver accounts must enforce a minimum password length of 12 characters with complexity requirements.
NFR-S-06	All API inputs must be validated and sanitized server-side to prevent SQL injection, XSS, and parameter tampering.

6.5 Portability & Scalability

ID	Requirement
NFR-Po-01	All backend microservices must be containerized using Docker and deployable via Kubernetes.
NFR-Po-02	The Kafka ingestion pipeline must be horizontally scalable by adding consumer instances without downtime.
NFR-Po-03	The mobile application must support Android 8.0+ (API Level 26) and iOS 14+.
NFR-Po-04	The web dashboard must be compatible with Chrome 100+, Firefox 100+, and Safari 15+ without polyfills.
NFR-Po-05	The route data model must accommodate an arbitrary number of stops and routes to support expansion beyond the MVP corridor.

7. System Constraints & Assumptions

7.1 Constraints

- The MVP system scope is limited to the Moratuwa-to-Kadawatha corridor. Multi-route support is an architecture requirement but will not be fully loaded until Phase 2.
- The Arduino Nano's 2KB SRAM imposes strict firmware memory management requirements. No dynamic memory allocation libraries (e.g., String class, vector) may be used in production firmware.
- The 2G cellular network (GPRS) maximum uplink bandwidth of approximately 85 kbps constrains payload size. Each MQTT payload must not exceed 512 bytes.
- Bus operators must grant permission and physical access for hardware installation. The project cannot proceed on a bus without written operator consent.
- The Keycloak IAM instance must be deployed in the same cloud region as the backend to meet JWT validation latency requirements.

7.2 Assumptions

- Each bus on the MVP route will be equipped with a 12V DC power supply capable of powering the hardware unit via a USB adapter or voltage regulator.
- Bus operators participating in the MVP will provide at least one Android device per bus for the Driver app.
- The NTC will provide official stop coordinates and route polyline data for the Moratuwa–Kadawatha corridor.
- Average passenger load per bus on the target corridor is approximately 60 passengers, used as the basis for crowd state threshold calculation.
- The 2G GPRS network will remain available for the majority of the route. Known dead zones will be handled by the offline buffering system.
- Historical GPS trip data for the target corridor will be collected during a minimum 4-week data collection pilot prior to ML model training.

8. Acceptance Criteria & Testing Strategy

8.1 MVP Acceptance Criteria

The following criteria must all pass before the MVP is considered complete and ready for pilot deployment:

#	Criterion	Requirement Ref.	Test Method
1	Hardware unit achieves GPS fix and transmits valid payload within 60 seconds of power-on in open-sky conditions	FR-G1.1, FR-G1.5	Field test: 10 consecutive cold starts
2	Crowd state is correctly classified for each threshold band with < 5% misclassification rate	FR-G1.2	Lab test: controlled entry/exit sequences
3	End-to-end latency from hardware transmission to passenger app update is ≤ 5 seconds under normal load	NFR-P-01, NFR-P-02	Integration test with timestamp comparison
4	XGBoost ETA model achieves MAE < 90 seconds on the holdout test set	FR-G2.2, NFR-P-05	Offline model evaluation on historical data
5	Anomaly detection flags a simulated off-route bus within 30 seconds	FR-G2.4	Integration test: GPS spoofing off-route
6	Bus state machine rejects all invalid state transitions	FR-G3.2	Unit test: all invalid transition permutations
7	Admin dashboard displays live bus positions within 5 seconds on a 50-bus simulation	FR-G4.3, NFR-P-02	Load test with simulated fleet data
8	All REST API authenticated endpoints return HTTP 401 when called without a valid JWT	NFR-S-01, NFR-S-02	Security test: API fuzzing with missing/invalid tokens
9	Offline buffering successfully stores and replays 30 minutes of GPS data after reconnection	FR-G1.4, NFR-R-04	Integration test: cellular network disconnection simulation

8.2 Testing Levels

- **Unit Testing:** All backend microservices shall achieve minimum 80% line coverage using automated unit test suites.
- **Integration Testing:** G1-to-G2 and G2-to-G3 data flows shall be validated using a fully simulated hardware environment with GPS replay files.
- **System Testing:** End-to-end system tests shall be executed in a staging environment that mirrors the production cloud configuration.
- **User Acceptance Testing (UAT):** A minimum of 20 representative passenger users and 5 drivers shall participate in a structured UAT session on the MVP route prior to public launch.
- **Performance Testing:** Load testing shall simulate 50 concurrent active buses and 500 concurrent passenger app connections.

- Security Testing: Penetration testing shall be conducted on all public-facing API endpoints prior to production deployment.

9. Appendix

Appendix A — Hardware Bill of Materials (BOM)

Component	Model / Spec	Unit Cost (LKR)	Purpose
Microcontroller	Arduino Nano v3 (ATmega328P)	950	Core processing unit
GPS Module	u-blox NEO-6M with antenna	1,400	Location acquisition
GSM/GPRS Module	SIMCom SIM800L	1,200	Cellular data transmission
Passenger Sensor x2	HC-SR04 Ultrasonic / IR pair	600	Crowd counting at doors
SD Card Module	SPI SD breakout	350	Offline data buffering
Voltage Regulator	LM2596 DC-DC Buck converter	280	12V to 5V for Nano/GPS
Enclosure & PCB	Custom ABS enclosure + PCB	900	Ruggedized housing
Miscellaneous	Cables, SIM, connectors	320	Assembly and SIM costs
Total BOM Cost		6,000	Below LKR 8,000 target

Appendix B — Technology Stack Summary

Layer	Technology	Version / Notes	Responsible Group
Hardware	Arduino Nano + C++ firmware	ATmega328P, 16MHz	G1
Messaging	Eclipse Mosquitto (MQTT)	v2.x	G4
Streaming	Apache Kafka	v3.x	G4
ML Engine	XGBoost + Python	v2.x, scikit-learn pipeline	G2
Anomaly ML	Isolation Forest (sklearn)	Deployed as microservice	G2
Backend API	Spring Boot / FastAPI	REST + WebSocket	G2
Auth / IAM	Keycloak	v22.x, JWT OIDC	G4
Database	PostgreSQL + Redis	Postgres for state; Redis for live feed cache	G2/G4

Mobile App	React Native	iOS 14+ / Android 8+	G3
Web Dashboard	React.js	Chrome/Firefox/Safari	G3
Infrastructure	Kubernetes on AWS / GCP	Docker containers, auto-scaling	G4

Appendix C — Document Revision History

Version	Date	Author	Summary of Changes
v1.0	March 2025	Project Team	Initial draft — sketch notes and requirements outline
v2.0	April 2025	Project Team	Full SRS and Business Plan — IEEE 830 compliant, complete requirements, API contracts, business financials, acceptance criteria

OnTime Public Transport Intelligence System — SRS v2.0
Confidential — For internal project use only. Not for public distribution.

Tab 2

3. Functional Requirements

Functional requirements are organized by subgroup. Each requirement is uniquely identified by its FR identifier for traceability.

3.1 G1 — IoT & Edge Hardware Requirements

G1 is responsible for all embedded firmware running on the Arduino Nano hardware units installed in each bus. The unit integrates the NEO-6M GPS receiver, SIM800L GSM/GPRS modem, IR passenger counting sensors, and an SD card module for offline buffering.

FR-G1.1 — GPS Data Acquisition

Priority: Critical

Module: NEO-6M GPS Receiver + Arduino Nano

The firmware must continuously parse NMEA 0183 GGA and RMC sentences from the NEO-6M GPS module via a hardware or SoftwareSerial UART connection. It must extract the following fields from each valid fix:

- Latitude and longitude (decimal degrees, 6 decimal places minimum)
- Ground speed in km/h
- GPS fix timestamp (UTC, ISO 8601 format)
- Fix quality indicator (must be 1 or 2; discard fixes with quality 0)

The firmware must poll the GPS module at a minimum rate of 1 Hz (one NMEA sentence parse per second) and must not block the main loop for longer than 800 ms during any GPS parse cycle.

FR-G1.2 — Crowd Density Sensing

Priority: High

Module: IR Obstacle Sensors (door-mounted) + Arduino Counter Logic

The firmware must maintain an integer passenger count variable by incrementing on door-entry IR beam breaks and decrementing on door-exit IR beam breaks. The count must be bounded between 0 and the bus maximum capacity (configurable constant, default: 60).

The firmware must map the integer count to one of three discrete crowd state values before including it in any outbound telemetry payload:

- NOT_FULL: 0 – 40% of capacity occupied
- SEMI_FULL: 41 – 75% of capacity occupied
- FULL: 76 – 100% of capacity occupied

Using discrete states rather than raw counts reduces payload size and protects occupancy data privacy.

FR-G1.3 — Local GPS Data Filtering

Priority: High

Algorithm: Moving-Average Filter (window size = 5 samples)

To eliminate GPS noise and signal drift artefacts that are common in urban canyon environments (such as along the Galle Road corridor), the firmware must apply a moving-average smoothing filter to the raw latitude and longitude values before incorporating them into outbound payloads.

The filter must detect and suppress GPS jumps where the computed instantaneous speed between two consecutive fixes exceeds 120 km/h (physically impossible for a bus on this route). Such anomalous readings must be discarded and the previous valid coordinate retained.

FR-G1.4 — Offline Telemetry Buffering

Priority: High

Storage Medium: MicroSD Card Module (SPI interface)

The firmware must detect a GPRS connection loss within 10 seconds of the first failed MQTT publish attempt. Upon detecting connection loss, the firmware must:

- Cease MQTT publish attempts immediately
- Open a timestamped log file on the SD card and begin writing buffered telemetry payloads as newline-delimited JSON records
- Continue buffering for up to 30 minutes; discard the oldest records if SD card utilization exceeds 80%

Upon GPRS reconnection, the firmware must batch-publish all buffered payloads to the MQTT broker in chronological order before resuming live publishing. Batch publishing must be rate-limited to 10 messages per second to avoid overwhelming the broker.

FR-G1.5 — Telemetry Transmission

Priority: Critical

Protocol: MQTT over TCP/IP via SIM800L GPRS

The firmware must publish a telemetry payload to the MQTT topic `transport/bus/{busId}/location` at a regular interval of every 3 to 5 seconds during active trip operation. Each payload must conform to the JSON schema defined in Section 4.1. The firmware must use MQTT QoS Level 1 (at-least-once delivery) to ensure payload delivery reliability.

FR-G1.6 — GPRS Reconnection

Priority: Critical

The firmware must automatically attempt to re-establish the GPRS connection within 10 seconds of detecting a connection failure. Reconnection must use an exponential backoff strategy with a maximum retry interval of 60 seconds to avoid network congestion. The firmware must not enter an unrecoverable state due to persistent network failure; normal buffering (FR-G1.4) must continue throughout.

FR-G1.7 — Device Authentication

Priority: High

Each hardware unit must authenticate with the MQTT broker using a unique device-specific client ID (format: `G-BUS-{unitSerial}`) and a pre-provisioned MQTT username/password credential stored in non-volatile EEPROM. Credentials must not be hard-coded in firmware source code.

3.2 G2 — Data Intelligence & Backend Requirements

G2 is responsible for the server-side data ingestion pipeline, ETA prediction engine, anomaly detection service, and the REST/WebSocket API layer consumed by G3.

FR-G2.1 — Telemetry Ingestion Pipeline

Priority: Critical

Stack: MQTT Broker → Apache Kafka → Stream Processing Service

The backend must subscribe to the MQTT topic `transport/bus/+/location` (wildcard subscription) to receive payloads from all registered bus units. Each received MQTT message must be validated against the canonical JSON schema (Section 4.1) before being forwarded to the Kafka topic `transport-telemetry-raw`. Invalid payloads must be routed to a dead-letter queue (`transport-telemetry-dlq`) for operator review and must not be passed to the ML pipeline.

The ingestion pipeline must sustain a throughput of at least 50 messages per second (corresponding to 50 concurrent buses transmitting at 1 Hz) with a Kafka consumer lag not exceeding 500 messages under normal load.

FR-G2.2 — ETA Prediction Model

Priority: Critical

Algorithm: XGBoost Gradient Boosting Regressor

For each active bus, the ETA service must produce a predicted remaining travel time (in seconds) to every downstream stop on the bus's assigned route. Predictions must be recalculated upon every new telemetry event received from that bus.

The XGBoost model must use the following feature set as inputs:

- Current bus speed (km/h) from live telemetry
- Current GPS coordinates (latitude, longitude)
- Time of day (hour, minute encoded as cyclic sine/cosine features)
- Day of week (encoded as integer 0–6)
- Route segment identifier (current segment between two consecutive stops)
- Historical average segment travel time (from training dataset, 90-day rolling window)
- Historical variance of segment travel time
- Current crowd status (NOT_FULL = 0, SEMI_FULL = 1, FULL = 2)

The model must be retrained offline on a monthly basis using accumulated telemetry data. The retrained model artifact must be deployable via a blue-green model swap with zero downtime.

FR-G2.3 — Anomaly Detection Service

Priority: High

The anomaly detection service must monitor each active bus continuously and automatically flag the following anomaly conditions:

- Stationary Bus Alert: A bus has transmitted valid telemetry (GPS fix is valid) but has reported a speed of < 2 km/h for more than 5 consecutive minutes. This may indicate a breakdown, terminus hold, or heavy traffic obstruction.
- Off-Route Deviation Alert: The Haversine distance between the bus's current GPS coordinates and the nearest point on its assigned route polyline exceeds 50 metres. This may indicate a diversion, wrong-turn, or GPS spoofing.
- Communication Loss Alert: No valid telemetry payload has been received from a bus for more than 3 minutes during an active trip window.

Each alert must be published to the Kafka topic transport-anomaly-alerts with a structured payload including busId, routeId, anomalyType, detectedAt (UTC timestamp), and a human-readable description string.

FR-G2.4 — ETA REST API

Priority: Critical

Endpoint: GET /api/v1/eta/{bus_id}/{stop_id}

The ETA API must return the latest predicted arrival time for a given bus at a given stop. The response must include:

- busId (string)
- stopId (string)
- etaSeconds (integer — remaining seconds to stop)
- etaMinutes (float — computed from etaSeconds, rounded to 1 decimal place)
- crowdStatus (string: NOT_FULL | SEMI_FULL | FULL)
- confidence (float 0.0–1.0 — model prediction confidence score)
- lastUpdated (ISO 8601 UTC timestamp)

The endpoint must respond within 200 ms at the 95th percentile under normal load (50 concurrent buses).

FR-G2.5 — Driver Delay Reporting API

Priority: Medium

Endpoint: POST /api/v1/driver/report-delay

Authenticated drivers must be able to submit a delay report containing the delay reason category (TRAFFIC | BREAKDOWN | ACCIDENT | OTHER) and an estimated delay duration in minutes. The backend must incorporate the reported delay as an additive offset to all downstream ETA predictions for that bus for the duration specified, after which predictions revert to model-only values.

FR-G2.6 — WebSocket Live Feed

Priority: Critical

Endpoint: WS /ws/live-feed

The backend must maintain a WebSocket server that broadcasts bus position updates to all connected frontend clients (Passengers and Admins). On each new telemetry event processed, a location update message must be emitted to all subscribers. The message must include busId, routeId, lat, lng, speed, crowdStatus, and timestamp. The server must support at least 1,000 concurrent WebSocket connections.

3.3 G3 — Frontend Application Requirements

FR-G3.1 — Passenger Live Map

Priority: Critical

Platform: React Native (iOS & Android)

The Passenger mobile app must display an interactive map (based on OpenStreetMap tiles) showing:

- All active buses on the MVP route as animated map markers, updated in real time via WebSocket (FR-G2.6)
- The full route polyline from Moratuwa to Kadawatha
- All named bus stops along the route as tappable markers

Bus marker positions must be updated with end-to-end latency of less than 5 seconds from the moment the Arduino reads the GPS coordinate to the map marker updating on screen. Smooth marker animation must interpolate between position updates.

FR-G3.2 — ETA and Crowd Display

Priority: Critical

When a passenger taps a bus marker or a stop marker, a bottom-sheet panel must appear displaying:

- The predicted ETA in minutes (retrieved via FR-G2.4), formatted as 'Arrives in X min'
- A colour-coded crowd indicator: Green (NOT_FULL — Seats Available), Amber (SEMI_FULL — Limited Seats), Red (FULL — Standing Only)
- The bus route identifier and direction
- A 'Last Updated' timestamp showing data freshness

ETA values must auto-refresh every 15 seconds while the bottom-sheet is visible. If the WebSocket connection is lost, the app must display a 'Reconnecting...' banner rather than showing stale data silently.

FR-G3.3 — Driver Mobile Application

Priority: High

Access: JWT-authenticated (Keycloak)

The Driver app must provide:

- A secure login screen that authenticates against Keycloak and stores the JWT securely in device keychain storage
- A 'Start Trip' button that registers the driver's bus as active on the backend and begins the trip session
- An 'End Trip' button that de-registers the bus from the active fleet
- A delay reporting form with selectable reason categories and estimated delay duration (FR-G2.5)
- A simple status indicator showing GPS connectivity and MQTT connection status of the on-bus hardware

FR-G3.4 — Admin Web Dashboard

Priority: High

Platform: React.js (web browser)

The Admin Dashboard must provide:

- A full-screen fleet map displaying all active buses in real time, with the same WebSocket feed as the Passenger app
- A collapsible alerts panel on the right sidebar that lists all active anomaly alerts (FR-G2.3) sorted by severity and recency
- Each alert card must display: bus ID, route, anomaly type, time detected, and an 'Acknowledge' button to dismiss

- A bus dispatch panel allowing admins to assign a driver to a route and mark a bus as active
- Fleet summary statistics bar: total active buses, total alerts today, average fleet speed

3.4 G4 — Platform & Security Requirements

FR-G4.1 — MQTT Broker Provisioning

Priority: Critical

Service: AWS IoT Core (MQTT) or self-hosted Mosquitto on EC2

The MQTT broker must support TLS 1.2+ encryption for all IoT device connections. Each device must authenticate using unique per-device credentials (FR-G1.7). The broker must be configured with topic-level ACLs preventing any device from publishing to another device's topic.

FR-G4.2 — Kafka Cluster Configuration

Priority: Critical

The Kafka cluster must be provisioned with a minimum of 3 broker nodes for fault tolerance. The transport-telemetry-raw topic must have a replication factor of 3 and 6 partitions to support parallel consumer processing. Message retention must be set to 7 days. The cluster must be accessible only from within the AWS VPC.

FR-G4.3 — Authentication & Authorisation (Keycloak)

Priority: Critical

Keycloak must be configured with a dedicated realm (transport-system) containing three roles: passenger (implicit, no token), driver, and admin. All backend REST API endpoints except GET /api/v1/eta/* must require a valid JWT bearing the appropriate role claim. Tokens must have an expiry of 1 hour; refresh tokens must have an expiry of 24 hours.

FR-G4.4 — Infrastructure as Code

Priority: Medium

All AWS infrastructure (VPC, EC2 instances, RDS, MSK Kafka, ECS containers) must be defined and provisioned using Terraform scripts stored in the project Git repository. Infrastructure changes must be reviewed and approved before application via terraform apply.

4. External Interface Requirements

4.1 Hardware–Backend Interface (G1 → G2)

The IoT hardware unit communicates with the cloud backend exclusively via the MQTT protocol over an encrypted TLS connection.

Protocol: MQTT v3.1.1 over TLS 1.2

MQTT Topic: transport/bus/{busId}/location

QoS Level: 1 (At-Least-Once Delivery)
Transmission Rate: Every 3–5 seconds per bus during active trip
Max Payload Size: < 200 bytes (compact JSON)

Canonical MQTT Payload Schema (JSON):

Field	Type	Example	Description
busId	String	"G-BUS-01"	Unique bus hardware unit identifier
routeId	String	"ROUTE-42"	Assigned route identifier for this trip
lat	Float (6 dp)	6.927100	Current latitude in decimal degrees (WGS84)
lng	Float (6 dp)	79.861200	Current longitude in decimal degrees (WGS84)
speed	Float	32.5	Ground speed in km/h from GPS fix
crowdStatus	Enum String	"SEMI_FULL"	Passenger density: NOT_FULL SEMI_FULL FULL
timestamp	ISO 8601 UTC	"2025-04-06T08:23:11Z"	UTC timestamp of the GPS fix on the hardware unit

4.2 Backend–Frontend Interface (G2 → G3)

4.2.1 WebSocket Live Feed

Endpoint: WS /ws/live-feed
Protocol: WebSocket (RFC 6455) over WSS (TLS)
Push Rate: On each telemetry event processed (approx. every 3–5 seconds per bus)
Message payload pushed to all connected clients on each bus update:

Field	Type	Description
busId	String	Bus unit identifier
routeId	String	Active route identifier
lat	Float	Current smoothed latitude
lng	Float	Current smoothed longitude
speed	Float	Current ground speed (km/h)
crowdStatus	Enum String	NOT_FULL SEMI_FULL FULL
timestamp	ISO 8601 UTC	Server processing timestamp

4.2.2 ETA REST Endpoint

Endpoint: GET /api/v1/eta/{bus_id}/{stop_id}
Authentication: None required (public endpoint)
Response Format: application/json
SLA: P95 latency ≤ 200 ms

4.2.3 Driver Delay Report Endpoint

Endpoint: POST /api/v1/driver/report-delay
Authentication: Bearer JWT (driver role)

4.2.4 Admin Fleet Endpoints

GET /api/v1/admin/fleet: Returns array of all active bus objects with latest telemetry
GET /api/v1/admin/alerts: Returns array of all unacknowledged anomaly alerts
POST /api/v1/admin/alerts/{id}/acknowledge: Marks an alert as acknowledged by the admin
POST /api/v1/admin/dispatch: Assigns a driver–bus–route combination and activates a trip

All Admin endpoints require a valid JWT bearing the admin role claim, validated by Keycloak.

4.3 Communication Interface Summary

Interface	Protocol	Security	Notes
Arduino → MQTT Broker	MQTT v3.1.1	TLS 1.2, Device Credentials	2G GPRS cellular; 3–5 s interval; QoS 1
MQTT Broker → Kafka	Internal bridge	VPC private subnet	Topic: transport-telemetry-raw
Kafka → ETA Service	Kafka consumer API	VPC private subnet	Consumer group: eta-prediction-cg
Backend → Mobile App	WSS (WebSocket)	TLS 1.2, optional JWT	1,000+ concurrent connections
Mobile App → REST API	HTTPS REST	TLS 1.2, JWT for auth routes	JSON request/response
Keycloak → Backend	HTTPS OIDC	TLS 1.2	JWT validation via JWKS endpoint

5. Non-Functional Requirements

5.1 Performance Requirements

ID	Requirement	Metric / Threshold	Rationale
NFR-P1	End-to-end telemetry latency (Arduino GPS read → Passenger app map update)	≤ 2.0 seconds (P95)	Core user experience requirement for 'real-time' perception
NFR-P2	ETA API response time	≤ 200 ms (P95) under 50 concurrent bus load	Supports smooth app UI interactions
NFR-P3	XGBoost inference time per bus per cycle	≤ 200 ms	Must fit within the 2-second end-to-end budget
NFR-P4	WebSocket map update lag (server push → marker move)	< 5 seconds	Defined in FR-G3.1; user-facing map responsiveness
NFR-P5	Kafka consumer lag under peak load	≤ 500 messages	Prevents pipeline backpressure and stale ETAs
NFR-P6	MQTT publish interval per bus	Every 3–5 seconds	Balances GPRS bandwidth with position update freshness

5.2 Reliability Requirements

ID	Requirement	Detail
NFR-R1	G1 GPRS auto-reconnect	Firmware must re-establish GPRS within 10 seconds of connection failure with exponential backoff up to 60 seconds.
NFR-R2	Backend service availability	G2 backend services (Kafka, ETA API, WS server) must maintain ≥ 99.5% monthly uptime, equating to ≤ 3.65 hours downtime per month.
NFR-R3	Offline telemetry preservation	G1 must buffer up to 30 minutes of telemetry to SD card without data loss during a GPRS outage.
NFR-R4	ML model fallback	If the XGBoost model fails to produce an inference within 500 ms, the system must fall back to the historical-average-only ETA and flag the response with a low-confidence indicator.
NFR-R5	WebSocket reconnection	The mobile app must automatically attempt WebSocket reconnection on disconnect, with a 3-second retry interval, up to 10 attempts before displaying an error state.

5.3 Security Requirements

ID	Requirement	Implementation
NFR-S1	Transport encryption	All network communications (MQTT, REST, WebSocket) must use TLS 1.2 or higher. Self-signed certificates are not permitted in production.

NFR-S2	IoT device authentication	Each Arduino unit must authenticate with a unique MQTT credential stored in EEPROM (not in firmware source). MQTT broker ACLs prevent cross-device topic access.
NFR-S3	API authentication & authorisation	All Admin and Driver API endpoints must validate the Bearer JWT signature, expiry, and role claim via Keycloak JWKS. Unauthenticated requests must receive HTTP 401.
NFR-S4	JWT security	Driver JWTs: 1-hour expiry. Admin JWTs: 1-hour expiry. Refresh tokens: 24-hour expiry. Tokens must be stored in secure device keychain (iOS Keychain / Android Keystore), never in plain SharedPreferences or AsyncStorage.
NFR-S5	Network isolation	All backend services (Kafka, Keycloak, RDS) must be deployed within an AWS VPC private subnet with no direct public internet access. Only the MQTT broker, API Gateway, and Keycloak authentication endpoint are publicly exposed.
NFR-S6	Data minimisation	Passenger location data must never be collected or transmitted. The app must not require any personally identifiable information from Passengers. Raw passenger counts must not be transmitted; only crowd state enum values.

5.4 Scalability Requirements

- **NFR-SC1 — Minimum Fleet Capacity:** The G2 backend pipeline must process telemetry and run ETA inferences for at least 50 concurrent active buses without degradation beyond defined performance thresholds.
- **NFR-SC2 — WebSocket Connections:** The WebSocket server must support at least 1,000 concurrent client connections (Passengers + Admins) without requiring horizontal scaling for the MVP.
- **NFR-SC3 — Horizontal Scalability:** The ETA inference service and REST API layer must be deployable as stateless containers (AWS ECS) to allow horizontal scaling beyond 50 buses in future releases with no architectural changes.
- **NFR-SC4 — Kafka Partition Strategy:** The transport-telemetry-raw topic must be partitioned by busId (6 partitions) so that adding more Kafka consumer instances for future bus fleets requires no topic reconfiguration.

5.5 Hardware Constraint Requirements

- **NFR-H1 — SRAM Optimisation:** All firmware data structures, MQTT payload buffers, and GPS parse buffers must fit within the Arduino Nano's 2 KB SRAM. String objects must use the F() macro to store string literals in Flash memory. Dynamic memory allocation (malloc/new) is prohibited in firmware.
- **NFR-H2 — SoftwareSerial Management:** The GPS module and GSM module must not share the same SoftwareSerial instance. If both must be active simultaneously, hardware UART must be allocated to the higher-priority module (GPS). Read intervals must be carefully managed to prevent UART buffer overflow.
- **NFR-H3 — Flash Memory:** Total firmware binary size must not exceed 28 KB (of the 30 KB available Flash on the ATmega328P), preserving 2 KB as a safety margin for future firmware updates.

- NFR-H4 — Power Consumption: The hardware unit must operate within a power budget of 500 mA at 5V (2.5W), sourced from the bus's 12V accessory circuit via a 7805 or equivalent voltage regulator.

5.6 Usability Requirements

- NFR-U1 — ETA Comprehension: The Passenger app must display ETAs in plain-language minutes (e.g., 'Arrives in 4 min') rather than raw seconds.
- NFR-U2 — Crowd Indicator Accessibility: Crowd status colours (Green / Amber / Red) must be accompanied by text labels ('Seats Available', 'Limited Seats', 'Standing Only') to comply with WCAG 2.1 Level AA colour-blindness accessibility guidelines.
- NFR-U3 — Offline State: The Passenger app must display a non-intrusive banner when the WebSocket connection is unavailable, and must clearly indicate that displayed data may be stale.

6. System Architecture Overview

The system follows a layered cyber-physical architecture with four distinct tiers: the Edge Layer (G1), the Intelligence Layer (G2), the Presentation Layer (G3), and the Platform Layer (G4). Data flows unidirectionally from the physical bus through the cloud pipeline to end-user interfaces.

6.1 Architecture Layers

Layer 1 — Edge (G1): Bus-Mounted IoT Unit

Each bus carries one Arduino Nano unit that continuously reads GPS coordinates from the NEO-6M module, estimates crowd density via IR sensors, applies local smoothing, and publishes JSON telemetry payloads to the cloud MQTT broker every 3–5 seconds over 2G GPRS via the SIM800L modem. If GPRS is unavailable, payloads are buffered to SD card and replayed upon reconnection.

Layer 2 — Intelligence (G2): Cloud Data Pipeline

The MQTT broker (AWS IoT Core) receives all telemetry payloads and forwards them to Apache Kafka (transport-telemetry-raw topic). Stream processing consumers read from Kafka, validate payloads, and fan out to two parallel services: the ETA Prediction Service (XGBoost inference) and the Anomaly Detection Service. Processed results are persisted to a PostgreSQL time-series database and pushed to the WebSocket broadcast layer.

Layer 3 — Presentation (G3): Client Applications

Three frontends consume the backend APIs: the Passenger mobile app (React Native, anonymous, WebSocket + REST), the Driver mobile app (React Native, JWT-authenticated, REST), and the Admin web dashboard (React.js, JWT-authenticated, WebSocket + REST). All map rendering uses the OpenStreetMap tile server with Google Maps SDK overlays.

Layer 4 — Platform (G4): Infrastructure & Security

All services are deployed on AWS (ap-south-1). Keycloak manages all identity and access. Terraform defines all infrastructure. A VPC with public/private subnet separation ensures that only the API Gateway, MQTT broker, and Keycloak authentication endpoints are reachable from the public internet.

6.2 Data Flow Summary

#	From	To	Data	Protocol
1	GPS Satellite	NEO-6M Module	NMEA 0183 sentences (GGA, RMC)	UART (serial)
2	Arduino Firmware	MQTT Broker (AWS)	Telemetry JSON payload (FR-G1.5)	MQTT/TLS over GPRS
3	MQTT Broker	Kafka (transport-telemetry-r-aw)	Raw telemetry message	Internal bridge
4	Kafka Consumer	ETA Prediction Service	Validated telemetry record	Kafka consumer API
5	Kafka Consumer	Anomaly Detection Service	Validated telemetry record	Kafka consumer API
6	ETA Service	PostgreSQL DB	Predicted ETAs per stop	JDBC / psycopg2
7	Anomaly Service	Kafka (transport-anomaly-alerts)	Anomaly alert object	Kafka producer API
8	Backend WS Server	Passenger & Admin Apps	Live bus position update	WSS (WebSocket)
9	REST API	Mobile / Web Apps	ETA, Fleet, Alert data	HTTPS/JSON

7. Use Cases

UC-01: Passenger Checks ETA at a Bus Stop

Actor: Passenger (anonymous)

Precondition: At least one bus is active on the Moratuwa–Kadawatha route; the app is open with a live WebSocket connection

Main Success Scenario:

1. Passenger opens the app. The Passenger map loads and displays all active bus markers on the route polyline.
2. Passenger taps the stop marker for their intended boarding stop (e.g., 'Dehiwala Bus Stop').

3. The app issues GET /api/v1/eta/{nearest_bus_id}/{stop_id} and displays the bottom-sheet ETA panel.
4. The panel shows 'Arrives in 6 min' with a green crowd indicator ('Seats Available').
5. The ETA auto-refreshes every 15 seconds as the bus approaches.

Alternative Flow: If no buses are active, the map shows an empty route with 'No active buses' message.

Postcondition: Passenger has accurate ETA and crowd information to make a boarding decision.

UC-02: Bus Driver Starts a Trip

Actor: Bus Driver (authenticated)

Precondition: Driver has valid Keycloak credentials; G1 hardware unit is powered and connected

Main Success Scenario:

6. Driver opens the Driver app and enters credentials. Keycloak validates and returns a JWT.
7. Driver taps 'Start Trip' and selects the assigned route (ROUTE-42: Moratuwa → Kadawatha).
8. App posts to /api/v1/driver/start-trip with the JWT and route ID.
9. Backend registers the bus as active; the bus marker appears on all Passenger and Admin maps.
10. G1 hardware begins publishing live telemetry; ETA predictions are computed from the first GPS fix.

Postcondition: Bus is visible to Passengers and Admins; ETAs are being computed and served.

UC-03: Driver Reports a Traffic Delay

Actor: Bus Driver (authenticated, active trip)

Precondition: Driver is on an active trip; bus is delayed due to heavy traffic on Galle Road

Main Success Scenario:

11. Driver taps 'Report Delay' in the app.
12. Driver selects delay reason 'TRAFFIC' and sets estimated delay of 10 minutes.
13. App posts to /api/v1/driver/report-delay with JWT, reason, and delay minutes.
14. Backend adds a 10-minute additive offset to all downstream ETA predictions for this bus.
15. All connected Passenger apps immediately receive updated ETAs via WebSocket push.

Postcondition: Passengers see updated ETAs reflecting the reported delay within 5 seconds.

UC-04: Admin Responds to an Off-Route Anomaly

Actor: Fleet Admin (authenticated)

Precondition: G2 anomaly detection has flagged bus G-BUS-07 as 120 metres off its assigned polyline

Main Success Scenario:

16. The Admin Dashboard alert panel displays a red alert card: 'G-BUS-07 — Off Route Deviation — 120 m'.
17. Admin taps on G-BUS-07's marker on the fleet map to view its current position.
18. Admin contacts the driver via external communication channel to investigate.
19. Admin taps 'Acknowledge' on the alert card to dismiss it from the active alerts list.
20. The acknowledged alert is archived with the admin's ID and timestamp in the database.

Postcondition: Alert is archived; the anomaly is logged for post-incident review.